

Анализ зависимостей веб-страницы

Иногда разработчики испытывают трудности с анализом зависимостей на своих страницах.

В этой статье рассмотрим два вопроса, которые возникают чаще всего:

- [Как анализировать пакеты?](#)
- [Как определить, кто притянул на страницу конкретный модуль?](#)

Но, прежде чем переходить к самим проблемам, опишем кратко процесс записи пакетов и создание специальных логов по ним.

Процесс записи пакетов и создание логов

Если необходимо определить, из-за каких модулей нам на страницу прилетел конкретный [кастомный пакет](#), нам понадобятся специальные логи.

Специальные логи формируются в виде:

<текущая зависимость> -> <кастомный пакет, в котором эта зависимость лежит>

Чтобы включить специальные логи, передайте в URL параметр **viewLogger** со значением **true**, например, <https://test-online.sbis.ru/?viewLogger=true>.

На серверной стороне ([Сервис представления](#), в случае VDOM, на сервере этим занимается логика из [SbisEnvUI/Bootstrap](#)) реализована логика, которая строит полное дерево зависимостей иницирующих модулей страницы (данные модули прописываются в [маршрутах](#) через **addDeps**) – только необходимый набор модулей для работы конкретной страницы.

Далее, для каждого модуля из набора "Сервис представления" анализирует, есть ли рассматриваемый модуль в кастомном пакете:

1. Если модуль есть:

- запишите кастомный пакет в набор **jsLinks** для js и **cssLinks** для css (это набор скриптов и линков соответственно, который впоследствии будет вставлен в разметку);
- не записывайте данный модуль в **rtpack**;
- запишите в лог информацию:

<текущая зависимость> -> <кастомный пакет, в котором модуль лежит>

2. Если модуля нет:

- запишите модуль в **rtpack**;
- в лог информация не попадает.



В конце сбора **rtpack** и составления **jsLinks** и **cssLinks** Сервис представления сохраняет **rtpack** на статике, и логи пакетов сохраняет следующим образом:

```
<полный путь до rtpack>.log
```

В случае VDOM вместо **rtpack** модули вставляются в разметку напрямую, а вместо **.log файла** логи пишутся напрямую в облако, и получать их придётся через **UUID** (id запроса клиента).

Как анализировать пакеты?

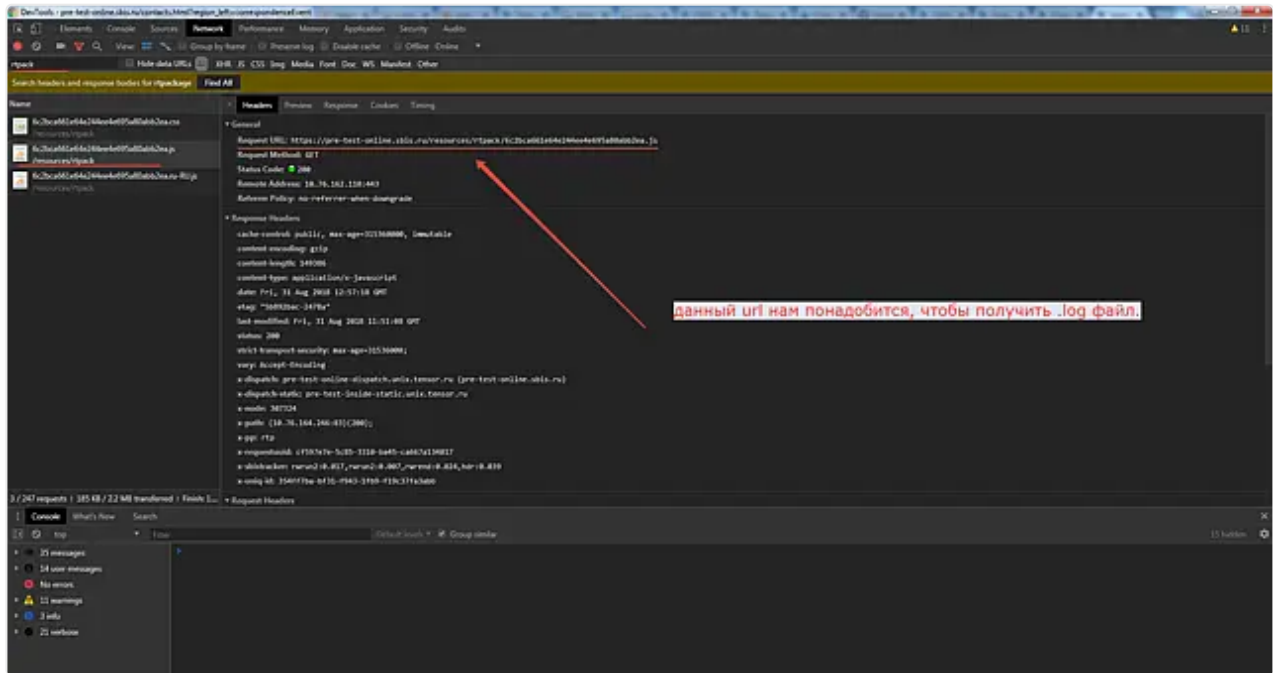
Прежде всего необходимо определить, какая по типу страница должна быть проанализирована: **обычная** или **VDOM**. У каждого типа страницы имеется своя схема получения логов по кастомным пакетам.

Анализ пакетов на обычных страницах

В качестве примера рассмотрим страницу "Контакты" на pre-test.

Необходимо выполнить следующие действия:

1. Зайдите на страницу pre-test-online.sbis.ru/;
2. Откройте **Dev Tools** и перезагрузите страницу, чтобы в **Network** присутствовал полный набор запросов. Настройте фильтр по **rtpack**. В итоге должна получиться следующая картина:



3. Скопируйте ссылку на js-ный **rtpack** (показано на скриншоте выше);
4. Добавьте в конце к ссылке **.log** и перейдите по полученному **url**. Для данного примера должна получиться [ссылка](#). Далее установится файл, откройте его.
5. Найдите в нем интересующий пакет, в качестве примера пусть это будет пакет **WS.Core/lib-compound-control.package.js**. Найденное количество покажет нам, сколько модулей из пакета требуются для работы конкретной страницы. Для рассматриваемого пакета должен получиться следующий результат:



Таким образом, мы определили все необходимые для работы зависимости для страницы "Контакты", из-за которых был запрошен **WS.Core/lib-compound-control.package.js**.

Анализ пакетов на страницах VDOM

Для анализа пакетов на страницах VDOM используйте статью [Анализ зависимостей](#) с использованием Wasaby Developer Tools.

Анализ пакетов через покрытие на обычных страницах

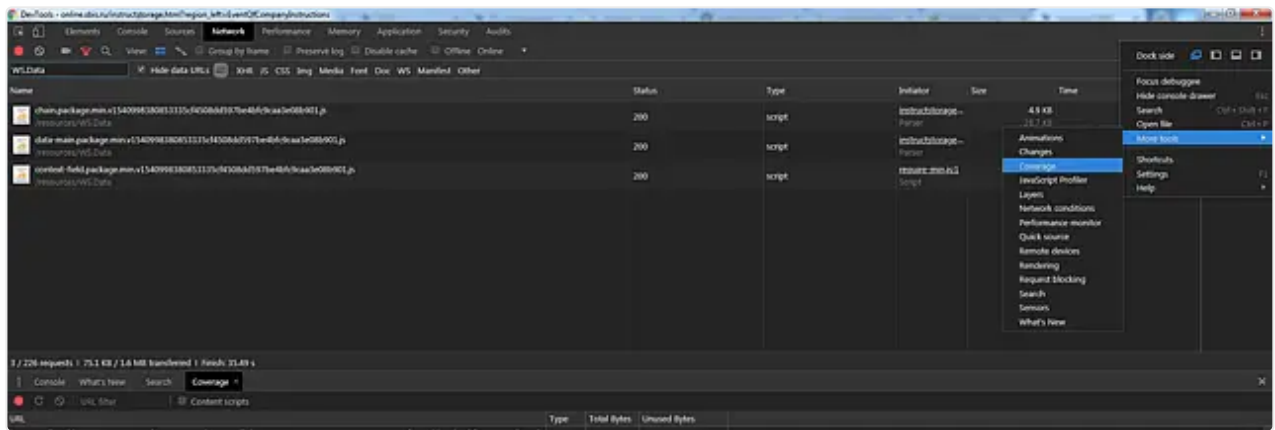
Как правило, при запросе любого конкретного модуля на сервер загружается весь пакет. В связи с этим, часть кода попросту не используется и загрузка ее бессмысленна.

Покрывтие показывает, какой процент кода используется на странице в текущий момент.

Чтобы приступить к анализу пакетов через покрытие, выполните следующие действия:

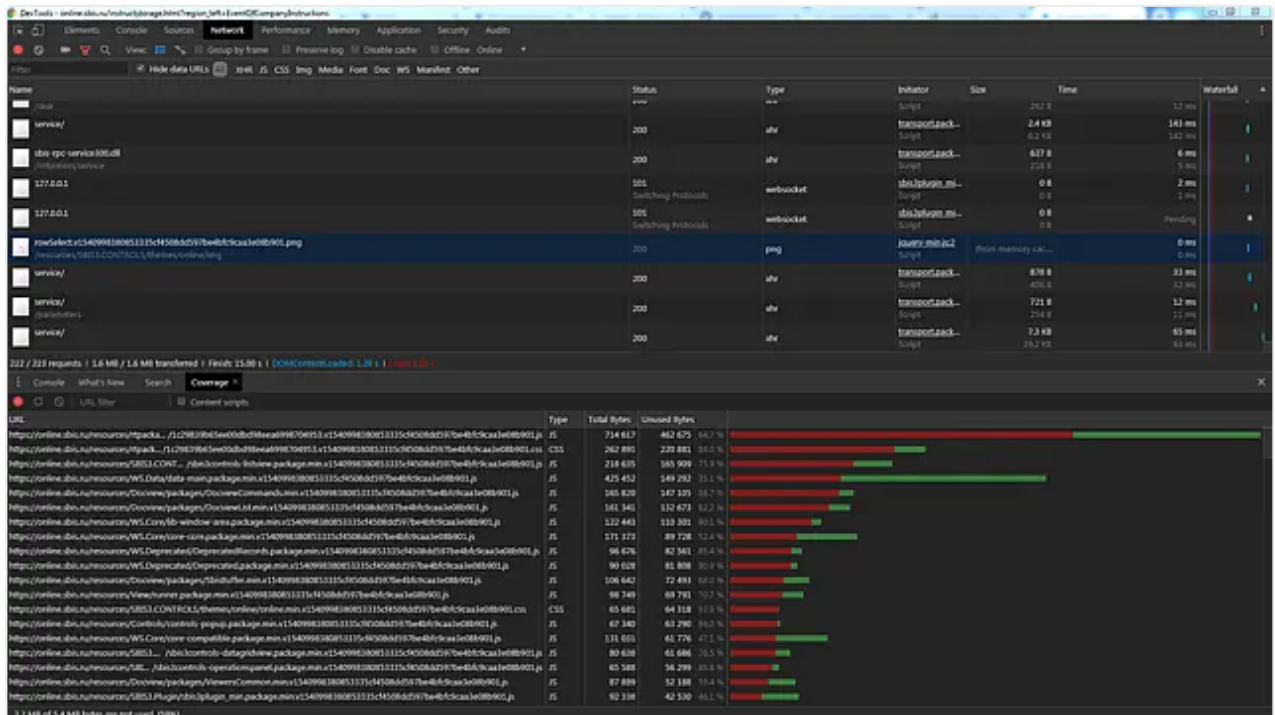
1. Откройте инструменты разработчика (F12), кликните на кнопку вызова панели инструментов в правом верхнем углу, выберите **More tools** → **Coverage**.

Coverage анализирует CSS/JS-файлы и сообщает о том, какая их часть используется на сайте в данный момент.

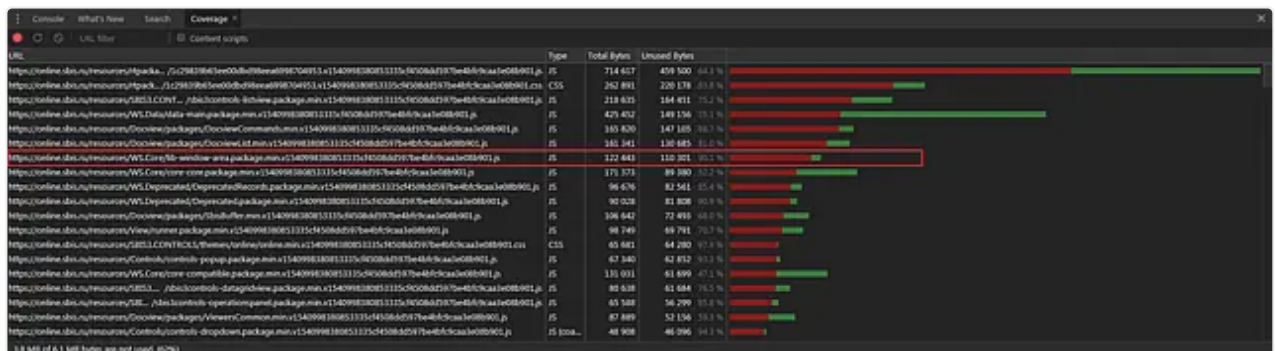


2. Для начала работы нужно нажать на круглую кнопку в левом верхнем углу вкладки Coverage. Когда она красного цвета, идёт анализ ресурсов и результаты автоматически обновляются.

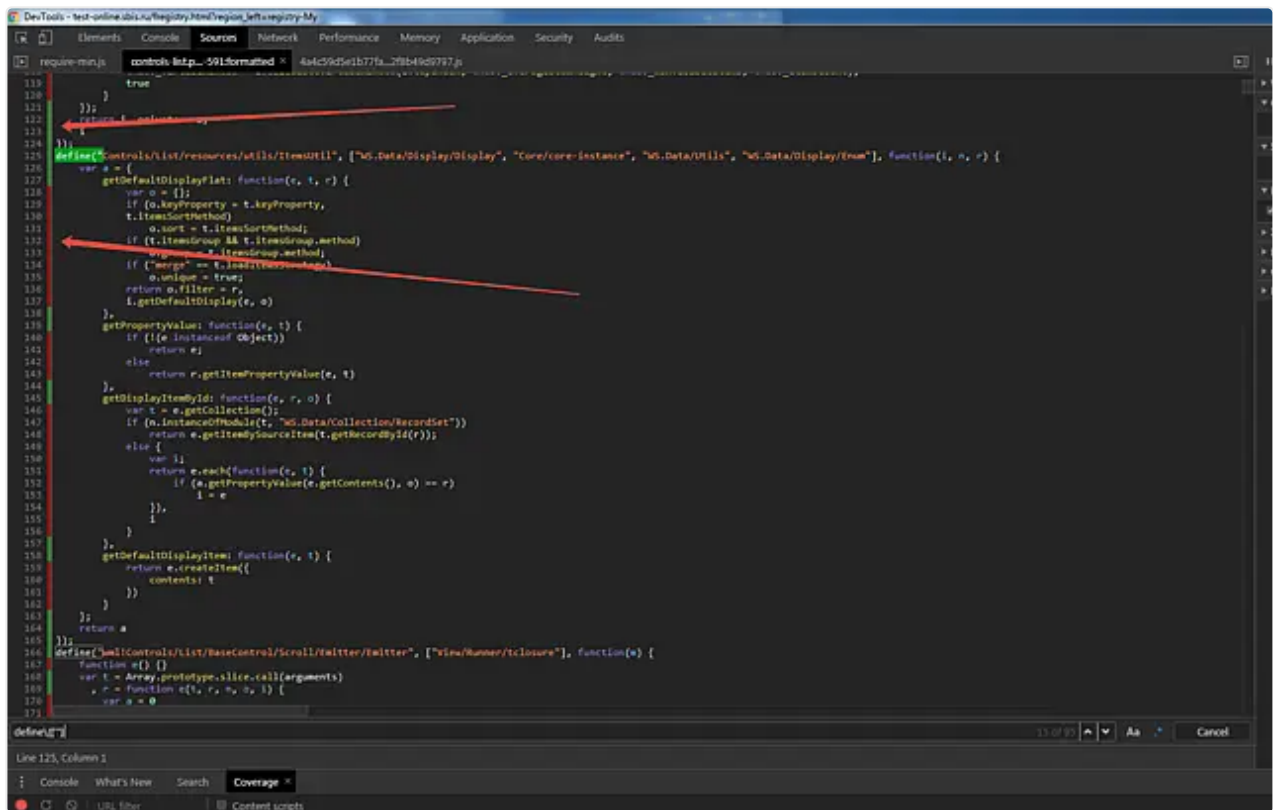
Запускаем и обязательно обновляем страницу.



3. Находим наиболее проблемные пакеты, оценивая шкалы справа.



Оценить, каков процент выполнения кода, можно по вертикальной шкале слева.



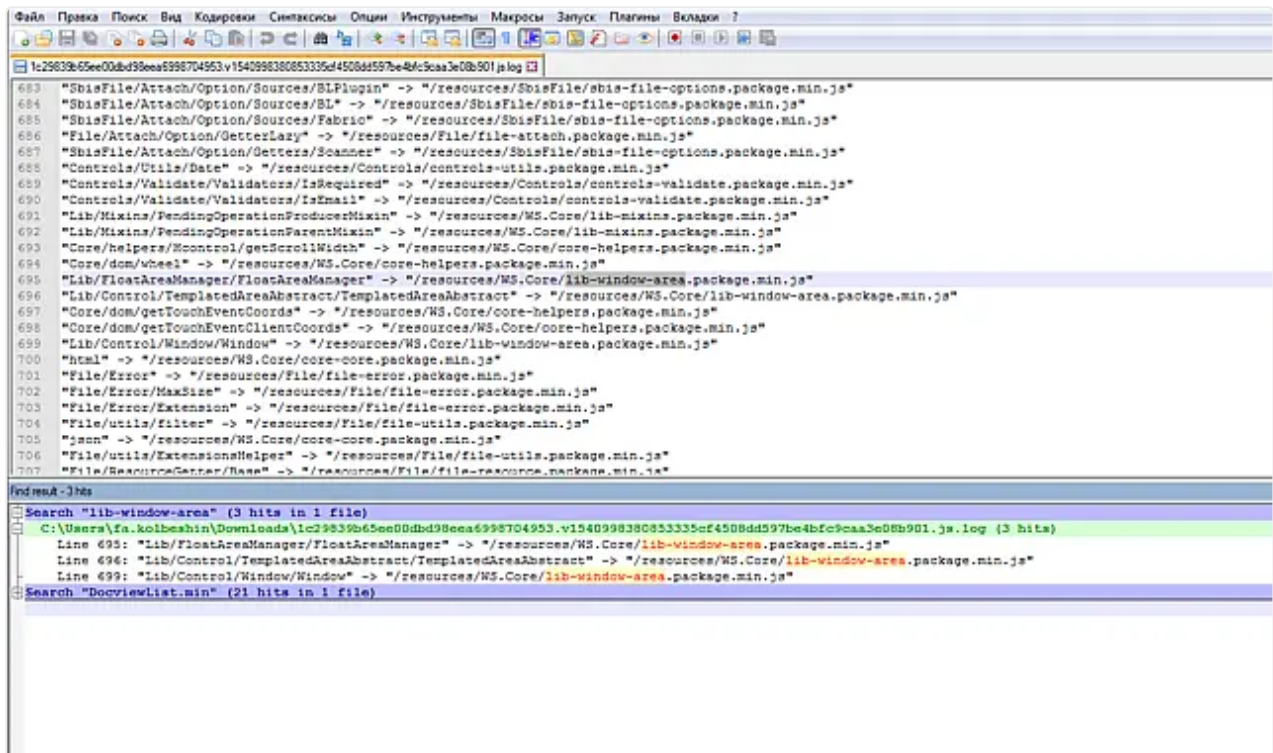
Красная шкала показывает код, который не исполнялся на текущий момент.

Код может не выполняться в случае, если модуль используется динамически, т.е. применяется при срабатывании определенного события/сценария, но подключен как явная зависимость, или был упакован в кастомный пакет, но в прогрузке страницы участия не принимал.

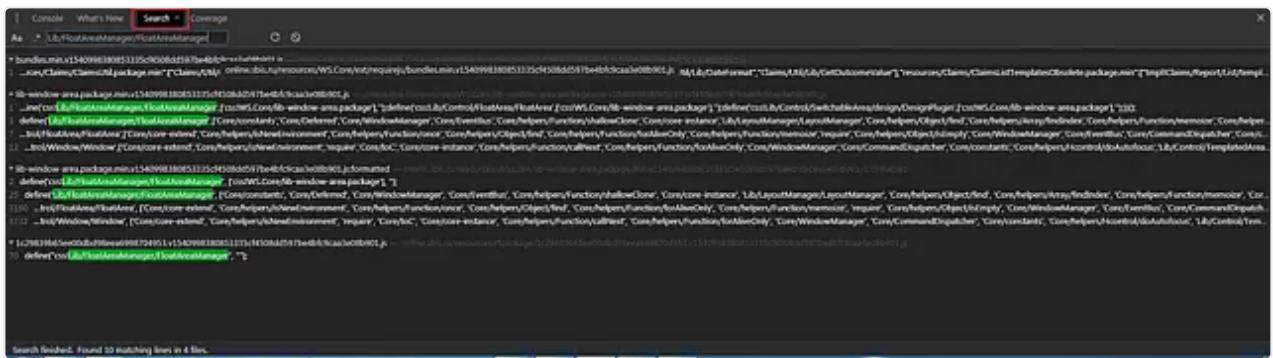
- Следующий шаг — поиск инициатора, который запросил пакет на страницу. Нужно определить, из-за каких модулей загрузился этот пакет. С этим нам поможет файл с логами, который для нас записал сервис представлений.

Подробнее об этом описано [здесь](#).

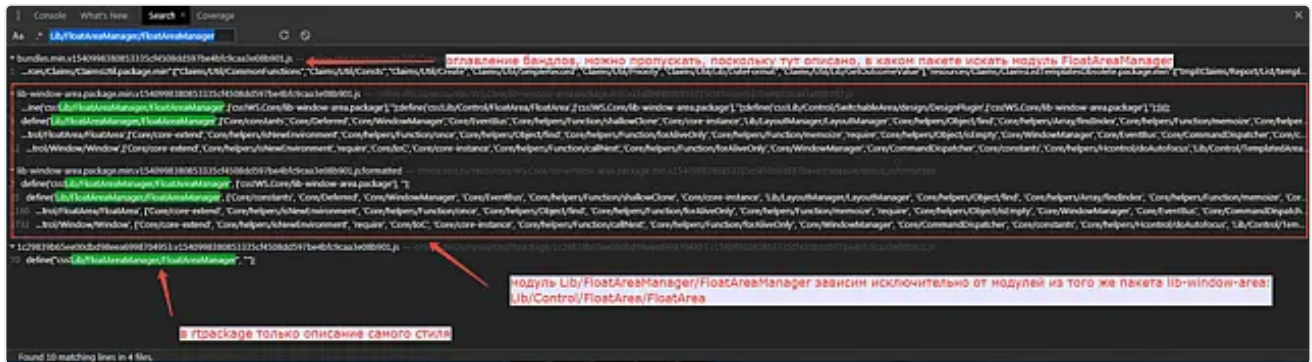
На примере пакета WS.Core/lib-window-area получаем следующий набор модулей:



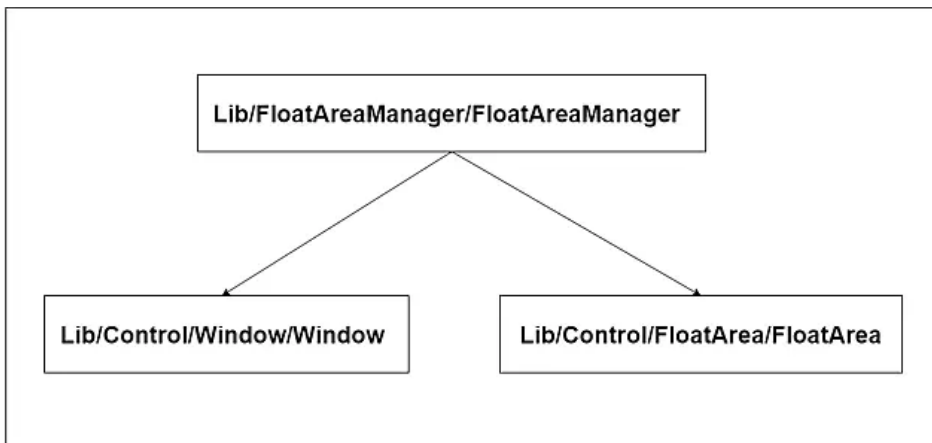
- После того, как мы выяснили, из-за какого модуля или модулей прилетел пакет, возвращаемся в Coverage и используем search, чтобы понять, кто затянул модули.



Рассмотрим на примере первой зависимости — `Lib/FloatAreaManager/FloatAreaManager`. Через Search находим использование этого модуля:



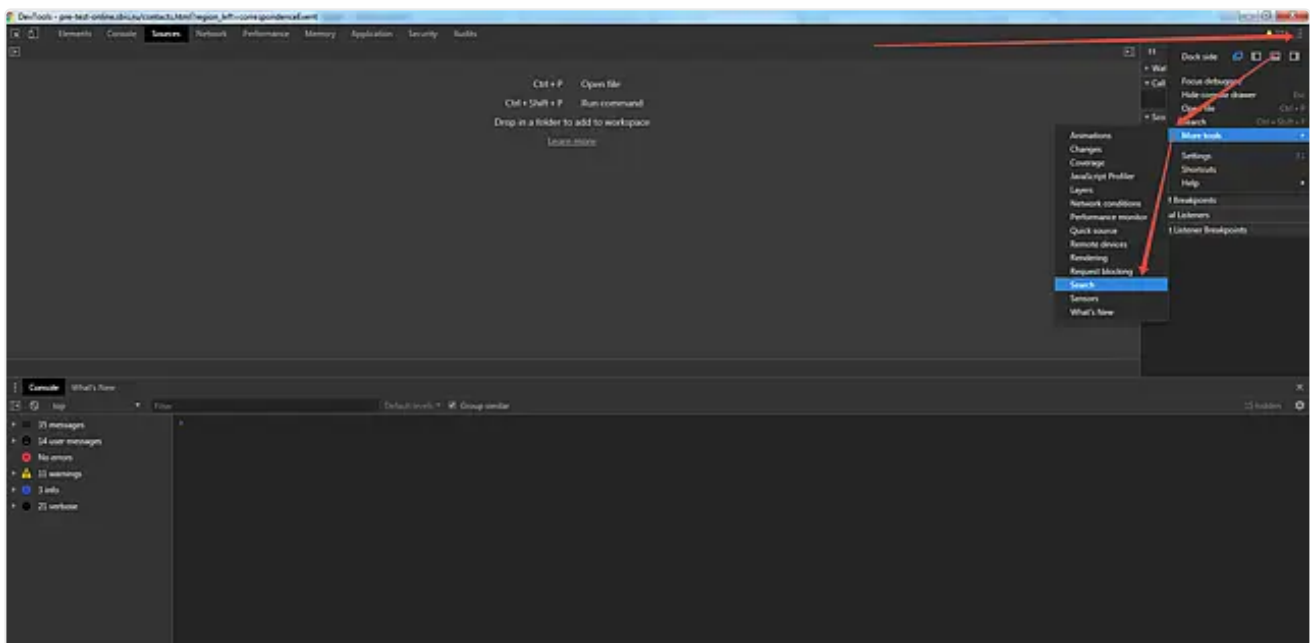
получаем следующую схему:



Если выполнить такое действие еще несколько раз, можно добраться до корневого элемента и выяснить, какое звено в этой цепи вызовов оказалось первым. Для `lib-window-area` это примет следующий вид:

```

graph TD
    Root["Lib/FloatAreaManager/FloatAreaManager"] --> Window["Lib/Control/Window/Window"]
    Root --> FloatArea["Lib/Control/FloatArea/FloatArea"]
    Window --> Dialog["Lib/Control/Dialog/Dialog"]
    Window --> LoadingIndicator["Lib/Control>LoadingIndicator/LoadingIndicator"]
    Window --> AreaAbstract["Lib/Control/AreaAbstract/ AreaAbstract.compatible"]
    Dialog --> ConsultantCommon["ConsultantCommon/Common"]
    Dialog --> PersonCoreDialogsConfirm["PersonCore/Dialogs/Confirm"]
    Dialog --> PersonCoreDialogsTextInput["PersonCore/Dialogs/TextInput"]
    PersonCoreDialogsConfirm --> InstructionsMixinsStorageListMixin["Instructions/Mixins/ StorageListMixin"]
    PersonCoreDialogsConfirm --> InstructionsListsList["Instructions/Lists/List"]
    PersonCoreDialogsTextInput --> PersonCoreDialogsConfirm
    PersonCoreDialogsTextInput --> PersonCoreDialogsConfirm
    FloatArea --> SBIS3_CONTROLS_FloatArea_FloatAreaSelector["SBIS3.CONTROLS/FloatArea/ FloatAreaSelector"]
    FloatArea --> Ellipsis1["..."]
    Ellipsis2["..."]
  
```



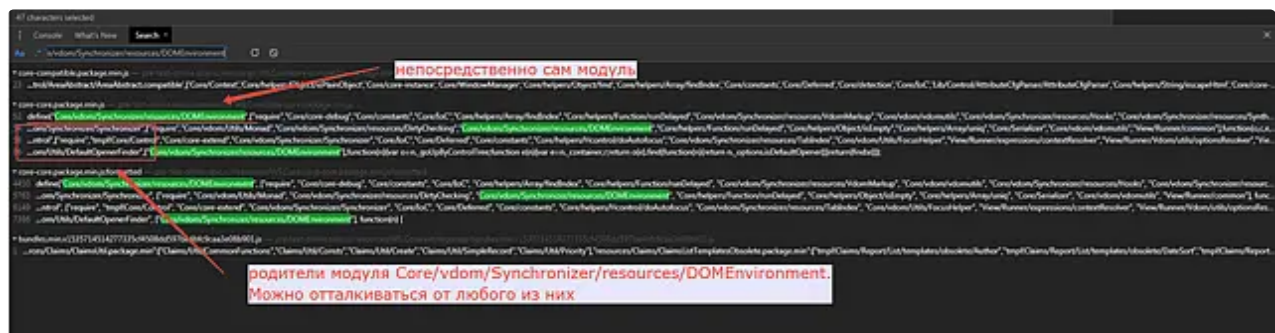
Выполните следующие действия:

1. Выберите интересующий модуль (например, для страницы pre-test-online.sbis.ru, которая на момент написания статьи является обычной, надо определить, почему прилетают модули неймспейса **Core/vdom**. Пусть это будет модуль **Core/vdom/Synchronizer/resources/Environment**);
2. Найдите через **Search** модуль **Core/vdom/Synchronizer/resources/Environment**:



Родителем является **Core/vdom/Synchronizer/resources/DOMEnvironment**.

Далее продолжайте поиск по **Core/vdom/Synchronizer/resources/DOMEnvironment**:



В результате имеем трех родителей, возьмите первого — **Core/vdom/Synchronizer/Synchronizer**.

Таким образом, цепочка снизу вверх сейчас имеет вид:

Core/vdom/Synchronizer/resources/Environment ->
Core/vdom/Synchronizer/resources/DOMEnvironment ->
Core/vdom/Synchronizer/Synchronizer

3. Повторяйте это действие, пока не дойдете до самого верха, когда родитель перестанет находиться.

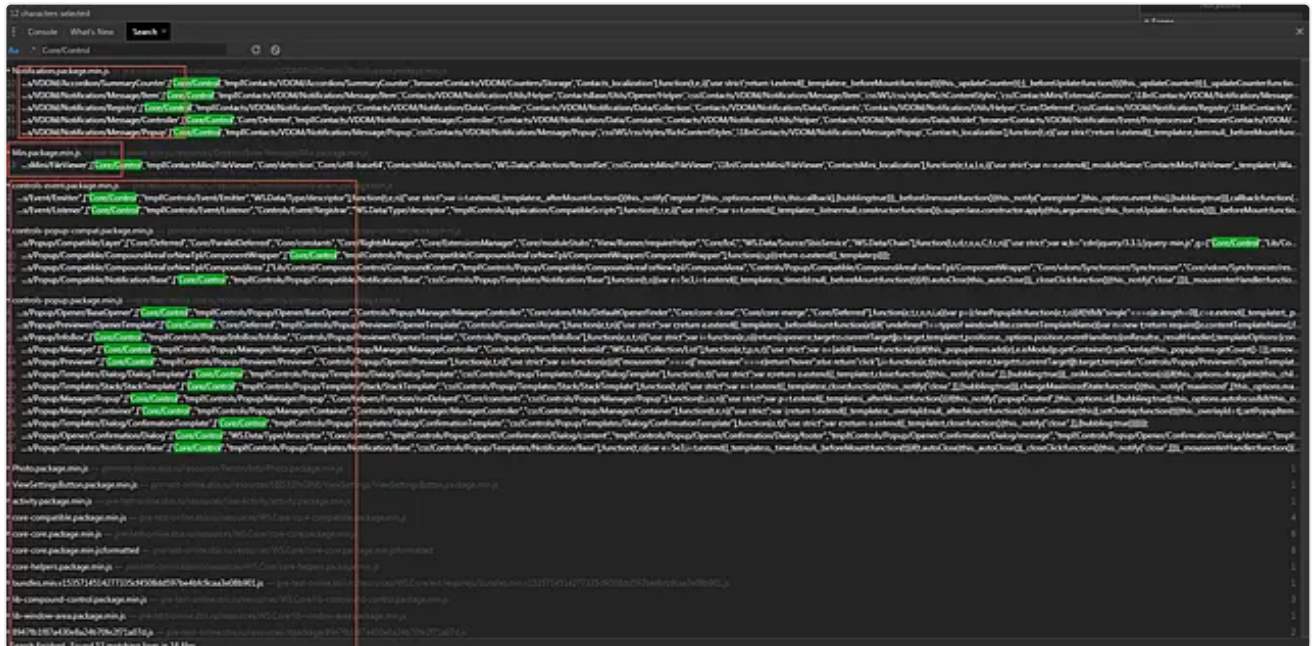
Это означает:

- либо мы дошли до корня и, в таком случае, мы нашли искомым модуль, а ответственные за свои страницы определяют, что это их модули;
- либо модуль был задефайнен в кастомном пакете, а сам пакет запросился не из-за этого модуля (проверить это можно по log-файлу).

В нашем примере, поднимаясь по дереву, мы получаем следующую цепочку:

```
Core/vdom/Synchronizer/resources/Environment ->
Core/vdom/Synchronizer/resources/DOMEnvironment ->
Core/vdom/Synchronizer/Synchronizer -> Lib/Control/Control
```

Соответственно, **Core/vdom** тянет **Lib/Control/Control**. Поднимаясь ещё выше, получаем следующую картину:



На основании картинки мы можем сделать вывод, что **Lib/Control/Control** является самым базовым компонентом, который используется как в платформенных компонентах, так и в прикладных.

В итоге, в рассмотренной ситуации убрать **Core/vdom** не получится, поскольку он необходим для работы базовых контролов.

В ситуации, когда у вас на странице конкретный кастомный пакет нужен только при выполнении определённого действия на загруженной странице, решением вашей проблемы будет **убрать модуль из списка явных зависимостей и запросить его через require внутри вашего модуля при определённой ситуации**.

Подобным образом и анализируются зависимости на страницах.

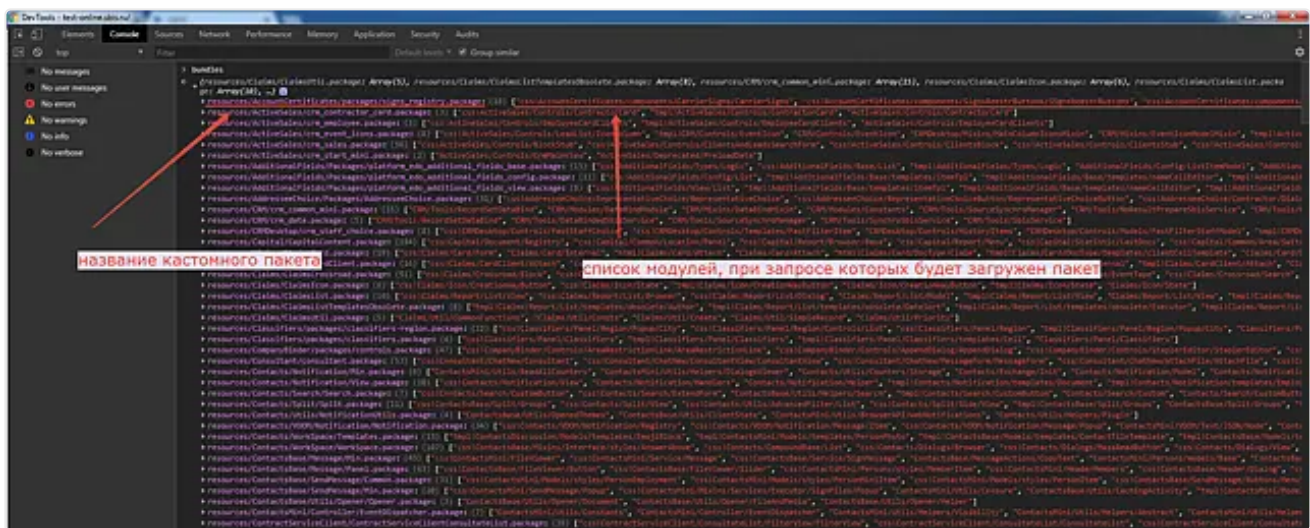
Также стоит отметить, что для пакетов и модулей существует целый набор мета-данных, которые также могут вам пригодиться для отладки.

Дополнительные мета-файлы, полезные для отладки

Мета-файл bundles.js

Данный файл содержит в себе всё оглавление кастомных пакетов, он используется для конфигурирования **requirejs**, чтобы он брал модули из пакетов.

К нему можно получить доступ из консоли вот так:



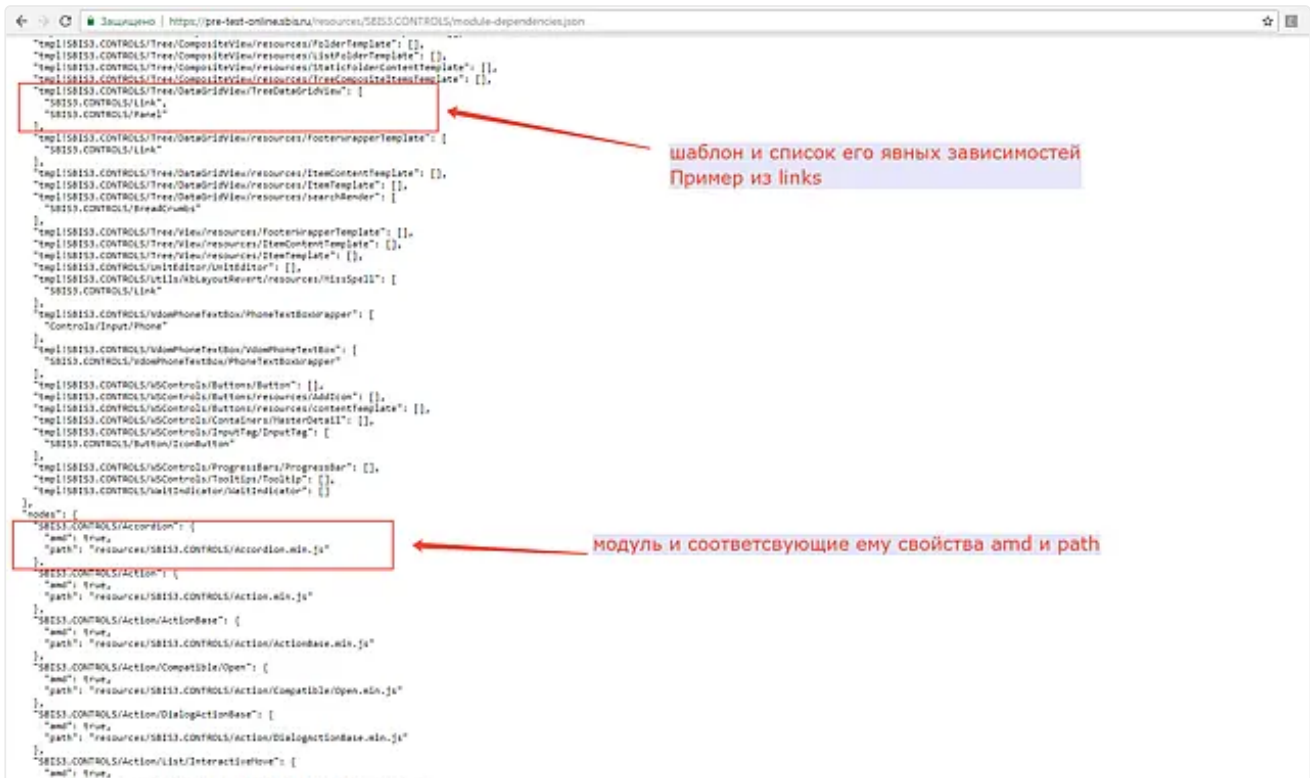
Файл доступен как глобальный объект **bundles**. Также можно запросить напрямую по пути `resources/WS.Core/ext/requirejs/bundles.js`.

Мета-файл `module-dependencies.json`

Данный файл содержит всё дерево зависимостей проекта.

Состоит из двух частей:

- **nodes** — узлы. Содержат информацию о пути до модуля, является ли модуль `amd`;
- **links** — зависимости. Содержат для каждого модуля информацию о явных зависимостях конкретного модуля.



Данный файл используется [Сервисом представления](#) для генерации `rpack`. Доступен `module-dependencies` как помодульно (например, для `SBIS3.CONTROLS` путь до него имеет вид `resources/SBIS3.CONTROLS/module-dependencies.json`), так и объединённый по пути `resources/module-dependencies.json`.

Мета-файл `bundlesRoute.json`

Данный json-файл построен по шаблону:

<модуль> -> <кастомный пакет, в котором этот модуль лежит>

Файл используется **Сервисом представления** для включения кастомных пакетов в разметку и исключения соответствующих модулей из `rt`-паковки.

[illegible]

По аналогии с **module-dependencies**, данный файл доступен как помодульно, так и объединённый.